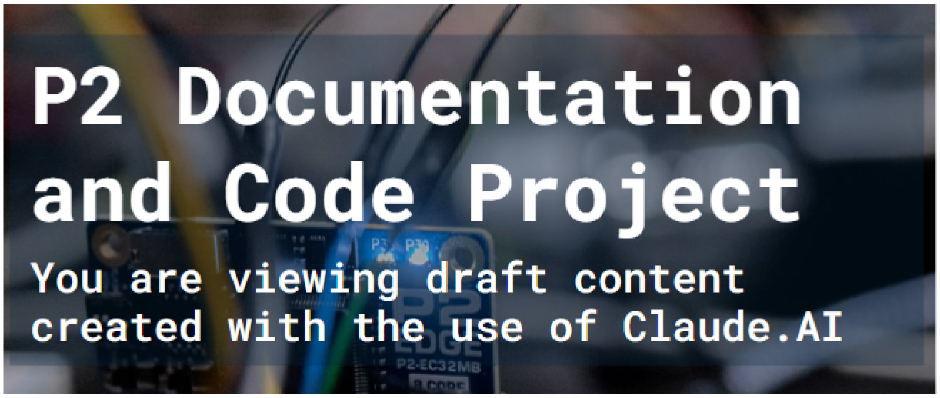

Propeller 2 • Application Note P2AN007

Data Structures with the New Language Facilities

Spin2 STRUCT for in-cog records — and the worked code for sharing them safely across cogs

July 2026

Version 1.0.0

What You'll Build

Real data structures on the P2 — a typed record with the Spin2 STRUCT facility, and four worked ways to move those records safely between cogs.

One shared idea — a STRUCT is a packed, named record; sharing it across cogs is about publishing it atomically — then a catalog of recipes you choose among by how the data flows:

- **R1 — in-cog record + array** — declare, fill, copy, and size a STRUCT
- **R2 — lock-free ring buffer** — one producer, one consumer, no lock
- **R3 — latest-wins mailbox** — a command block published with a sequence counter
- **R4 — locked multi-writer queue** — many writers, one hardware lock
- **R5 — a whole record in one long** — member bitfields; the publish IS the payload
- **R6 — raw addressing with OFFSETOF** — computed offsets, never hand-counted

Applies to: P2 (Propeller 2) silicon • Spin2 / PNut v45 or later (STRUCT; R6 needs v53, R5 needs v54) • P2 Edge or P2 Eval board; every recipe reports over DEBUG.

Iron Sheep Productions, LLC

P2 Knowledge Base Project

Abstract

Spin2's `STRUCT` facility gives the P2 real records — named, typed, packed fields you access by name instead of juggling parallel arrays and hand-computed offsets. Within one cog that's a convenience. Across cogs it becomes a design question: how do you hand a multi-field record from one cog to another without the reader ever seeing a half-written value? This note answers both. It shows how to declare and use a STRUCT, then works three concrete ways to move records safely between cogs — a lock-free ring buffer, a latest-wins mailbox, and a lock-guarded multi-writer queue — and closes with the two facilities added since: member bitfields, which can collapse a whole record into one atomically-publishable long, and `OFFSETOF`, which keeps raw addressing honest. It teaches the *implementation*; the question of *which* structure a given design should use, and why, belongs to the **P2 Architect's Guide** (a companion P2 Knowledge Base publication), which this note points to rather than repeats.

What you'll build: six small programs — an in-cog record and array, a single-producer/single-consumer ring buffer, a command mailbox published with a sequence counter, a multi-writer queue guarded by a hardware lock, a whole command record packed into a single long, and a packed wire header addressed by computed offsets.

Prerequisites & Hardware

You should already be comfortable with:

- Reading and writing Spin2 — methods, `VAR`/`DAT` blocks, and `DEBUG()`.
- Launching a method in another cog with `cogspin` (the cross-cog recipes run a second cog).
- The P2's hub RAM as the memory all cogs share, and the idea that a single long is read or written atomically while a multi-long update is not.

Version gate. STRUCT is a Spin2 language feature from **PNut / Spin2 v45**, so recipes R1–R4 open with `{Spin2_v45}` as their first line. Two later additions extend the facility and are covered here in their own section: `OFFSETOF` (**v53**, recipe R6) and member bitfields (**v54**, recipe R5). Each raises the floor only for the file that uses it — if your compiler predates them, R1–R4 are unaffected.

Where the design decision lives. This note is about the *how* — the worked code for each structure. The *which* — which data-flow contract a given seam should use, and why (latest-wins vs. queue vs. published telemetry, copy vs. reference) — is a decomposition decision covered by the **P2 Architect's Guide** and the knowledge base's data-flow-contracts model. When you're choosing, start there; when you're building the choice, this note has the code.

Hardware:

- A P2 Edge or P2 Eval board. Every recipe reports over DEBUG, so you can watch each work with no extra parts.
- Build with DEBUG enabled — `pnut_ts -d`.

The Idea

A `STRUCT` is a **packed record**: a named group of members — `BYTE`, `WORD`, `LONG`, nested structs, or arrays — laid out with no padding, allocated only when you declare a variable of that type. You define the type in a `CON` block and use it like any other type:

```
CON
  STRUCT reading_t(LONG timestamp, LONG value, BYTE status)  ' 9 bytes
```

Within one cog, that's the whole story: fewer bugs than parallel arrays, and the compiler tracks the layout for you. **Across cogs, one fact drives everything: a single long is read or written atomically, but a multi-field record is not.** If a producer writes a three-long record field by field while a consumer reads it, the consumer can catch the record half-updated — a *torn read*. Every cross-cog recipe here is a different answer to that one problem, and each answer comes down to the same move: **do all the multi-long writing first, then flip a single long that publishes it.** That single-long flip is atomic, so the reader either sees the old state or the fully-new one, never a mix.

There are two shapes of that move, and they map to when you need a lock:

- **One writer** of a given index or sequence long → **no lock needed.** The writer fills the record, then advances a single index (or bumps a sequence counter); the reader watches that one long. R2 and R3 are this.
- **Several writers** of the same index → **a lock is needed**, because "advance the index" is itself a read-modify-write that two writers can interleave. A P2 hardware lock makes that section exclusive. R4 is this.

How It Works (in brief)

Declaring and using a STRUCT. Define the type in `CON`; declare variables, arrays, or pointers of it; access members with dot notation.

You write	Meaning
`STRUCT point(x, y)`	a record with two LONG members (the default type)
`STRUCT rec(WORD a, BYTE b)`	typed members — BYTE, WORD, LONG
`point p` / `point ps[10]`	a variable / an array of records
`p.x := 100`	member access by name
`a := b`	copy an entire record (one statement)
`SIZEOF(point)`	the record's size in bytes

A record passes by value up to 15 longs; larger ones are copied with `BYTEMOVE` under the hood. One cross-object caution matters when a *child object* returns a record pointer — see the Pitfalls.

The atomic-publish discipline. A single hub long is the unit of atomicity. So the cross-cog pattern is always: write the record's fields, *then* write the one long that makes it visible — an index the reader compares, or a sequence counter it edge-detects. The reader never touches the record until it sees that long change. Get the *order* right (fields first, publish last) and a single-writer path needs no lock at all.

Choosing a Recipe

Pick by how the data flows — but see the Architect's Guide for the *why* behind the choice:

If the data flow is...	Use	Lock?
all within one cog	**R1** — in-cog record + array	no
one producer → one consumer, streaming	**R2** — lock-free ring buffer	no
one writer, "act on the newest," override old	**R3** — latest-wins mailbox	no
several writers → one queue	**R4** — locked multi-writer queue	yes (one hardware lock)
one writer, "act on the newest," **and the payload fits in 32 bits**	**R5** — whole record in one long	no

R5 is the strictest form of R3 and needs Spin2 v54; reach for it whenever the payload really does fit. **R6** is not a data-flow choice at all — it's the technique for reaching into a record from raw addresses (inline PASM, a buffer off the wire) without hand-counting offsets.

Each recipe is a complete, runnable program, and each ends with a ♦ **Verify**.

Recipe R1 — In-Cog Record + Array

Use it when you have related fields that belong together — a sensor reading's timestamp, value, and status — and want them as one named record instead of three parallel arrays.

```
{Spin2_v45}
CON
  _clkfreq = 200_000_000
  LOG_SIZE = 8

  ' A record type: three named members, packed (9 bytes, no padding).
  STRUCT reading_t(LONG timestamp, LONG value, BYTE status)

VAR
  reading_t log[LOG_SIZE]   ' an array of records, in this cog's memory
  LONG head

PUB main() | i, reading_t r
```

continues on next page →

↪ continued from previous page

```
    head := 0
    ' fill a rolling log of records
    repeat i from 0 to LOG_SIZE - 1
      r.timestamp := GETCT()                ' set members by name
      r.value     := i * i
      r.status    := i & 1
      log[head] := r  ' whole-struct copy into the array slot
      head := (head + 1) // LOG_SIZE

    ' read them back by member
    repeat i from 0 to LOG_SIZE - 1
      DEBUG("log[", udec(i), "] t=", uhex_long(log[i].timestamp), ...
            " v=", sdec(log[i].value), " s=", uhex(log[i].status))

    DEBUG("sizeof(reading_t) = ", udec(SIZEOF(reading_t)), " bytes")
```

How this works. `reading_t` groups three fields into one 9-byte record (`LONG` + `LONG` + `BYTE`, packed with no padding). `log[LOG_SIZE]` is an array of those records. You fill a local `r` by member, then `log[head] := r` copies the whole record into the slot in one statement — no field-by-field copying. Reading back, `log[i].value` reaches straight into the record. `SIZEOF(reading_t)` reports the packed size the compiler computed. This is the foundation the cross-cog recipes build on: a record you can copy and index as a unit.

★ **Tip — records replace parallel arrays.** Anywhere you'd keep `timestamps[]`, `values[]`, and `statuses[]` in lockstep, one `reading_t log[]` keeps them together and copies as a unit — fewer indices to keep aligned, fewer bugs.

♦ **Verify:** DEBUG prints eight `log[i]` lines with increasing `value` (`i*i`: 0, 1, 4, 9, …) and alternating `status`, then `sizeof(reading_t) = 9`. If the size prints as something other than 9, check the member types — a stray `LONG` where you meant `BYTE` changes the packed size.

Recipe R2 — Lock-Free Ring Buffer (One Producer, One Consumer)

Use it when one cog streams records to exactly one other cog — samples, events, log entries. With a single producer and a single consumer, no lock is needed at all.

```
  {Spin2_v45}
  CON
    _clkfreq = 200_000_000
    QSIZE = 16  ' power of two -> the & mask below is fast modulo
    QMASK = QSIZE - 1

    STRUCT sample_t(LONG seq, LONG value)  ' element type carried by the ring

  VAR
    sample_t queue[QSIZE]                  ' shared hub ring of records
    LONG qhead, qtail  ' producer OWNS head, consumer OWNS tail
```

continues on next page →

↪ continued from previous page

```
DAT
  prodStack   LONG 0[64]

PUB main() | sample_t s
  qhead := 0
  qtail := 0
  cogspin(NEWCOG, producer(), @prodStack) ' producer runs in its own cog

  ' Consumer (this cog): lock-free because only it writes qtail
  ' and only the producer writes qhead.
  repeat
    if qtail <> qhead                     ' something to take?
      s := queue[qtail]                   ' copy the record OUT first
      ' THEN publish the advance (single atomic long)
      qtail := (qtail + 1) & QMASK
      DEBUG("consumed seq=", udec(s.seq), " value=", sdec(s.value))

PRI producer() | n
  n := 0
  repeat
    if ((qhead + 1) & QMASK) <> qtail     ' not full?
      queue[qhead].seq   := n             ' fill the record FIRST
      queue[qhead].value := n * 10
      ' THEN publish head (single atomic long) -> element visible
      qhead := (qhead + 1) & QMASK
      n++
```

How this works. The ring is `QSIZE` records in shared hub memory, with a `QSIZE` that's a power of two so `& QMASK` is a one-instruction modulo. The safety comes entirely from **ownership**: the producer is the *only* cog that writes `qhead`, and the consumer is the *only* one that writes `qtail`. Each is a single long, written atomically. The producer fills a record's fields, *then* advances `qhead`; the consumer never looks at a slot until `qhead` has moved past it, so it can't see a half-written record. Same on the other side: the consumer copies the record out, *then* advances `qtail`. No lock, no torn reads — just correct ordering and single-owner indices.

▲ **Pitfall — publish the index last, always.** The whole guarantee rests on the producer writing the record *before* advancing `qhead`. Reverse those two lines and you publish the slot before its contents are written — the consumer reads stale or partial data. The single-writer-per-index rule is what lets you skip the lock; the write-then-publish order is what makes it correct.

♦ **Verify:** DEBUG prints a stream of `consumed seq=N value=M` lines with `value` = `seq * 10` and `seq` climbing monotonically. If you ever see a `value` that doesn't match its `seq * 10`, the publish order is wrong (index advanced before the fields were written). A stall (nothing printed) means head and tail started unequal — both must init to 0.

Recipe R3 — Latest-Wins Mailbox

Use it when one cog issues commands to another and only the *newest* matters — a target position, a mode, a setpoint. Old unread commands should be overwritten, not queued. A sequence counter published last makes it lock-free.

```
{Spin2_v45}
CON
  _clkfreq = 200_000_000

  STRUCT cmd_t(LONG opcode, LONG arg0, LONG arg1)  ' command's arg block

VAR
  cmd_t cmd  ' shared command block (one writer, one reader)
  LONG cmdSeq, cmdAck  ' writer bumps seq LAST; reader writes ack

DAT
  workerStack LONG 0[64]

PUB main() | i
  cmdSeq := 0
  cmdAck := 0
  cogspin(NEWCOG, worker(), @workerStack)

  repeat i from 1 to 5
    cmd.opcode := i                       ' fill the arg block FIRST...
    cmd.arg0   := i * 100
    cmd.arg1   := i * 1000
    ' ...then publish LAST (single atomic long) -> command visible
    cmdSeq++
    repeat until cmdAck == cmdSeq  ' wait for the worker to ack this one
    waitms(10)
  DEBUG("all commands acknowledged")

PRI worker()
  repeat
    if cmdSeq <> cmdAck                   ' a new command is published?
      execute(cmd.opcode, cmd.arg0, cmd.arg1)
      cmdAck := cmdSeq                    ' acknowledge (single atomic long)

PRI execute(op, a0, a1)
  DEBUG("worker exec op=", udec(op), " a0=", udec(a0), " a1=", udec(a1))
```

How this works. The command is a record; two extra longs carry a hand-off. The writer fills `cmd`'s fields, then increments `cmdSeq` — the atomic publish. The worker watches for `cmdSeq <> cmdAck`, runs the command, and writes `cmdAck := cmdSeq` to acknowledge. Because the sequence counter is bumped *after* the fields are written, the worker only ever executes a fully-formed command. This is "latest-wins": if the writer overwrote `cmd` twice before the worker looked, the worker would run the newest — old commands are replaced, not queued.

▲ **Pitfall — the ack is load-bearing; don't just delete it.** It looks like pacing, and it is tempting to drop so the writer never waits. But the worker reads `cmd.opcode`, `cmd.arg0`, and `cmd.arg1` as **three separate reads of shared memory**, and the ack is the only thing keeping the writer from overwriting the command *while the worker is part-way through reading it*. Delete it and your correctness quietly comes to rest on a race: whether the worker finishes all three reads before the writer's next write lands. A worker that does nothing but poll will usually win that race, which is exactly what makes this dangerous — it will *look* fine. Give the worker anything to do between the reads (a `CASE cmd.opcode` dispatch before it touches the arguments, say) and the writer starts landing inside the gap. If you need a writer that never blocks, take one of the two honest options: pack the payload into a single long so the whole record publishes in one store (**R5**), or have the reader re-check the sequence — read `cmdSeq`, copy the record, read `cmdSeq` again, and retry if it moved, so a copy that straddled an update is discarded rather than used.

★ **Tip — this replaces a "command pending" flag.** An older idiom has the writer spin on a flag the reader clears. The sequence-counter form is better: the seq/ack pair makes "which command are we on" explicit rather than implicit in a single boolean, and it extends naturally to the non-blocking re-check above.

♦ **Verify:** DEBUG shows five `worker exec op=N ...` lines in order (1..5), each with `a0 = op * 100` and `a1 = op * 1000`, then `all commands acknowledged`. If the worker ever runs a command with mismatched args, the fields were published after the sequence bump — put the `cmdSeq++` *last*.

Recipe R4 — Locked Multi-Writer Queue

Use it when *several* cogs feed one queue — multiple producers, one drain. Now "advance the head" is a read-modify-write two writers can interleave, so a P2 hardware lock makes the enqueue exclusive.

```
{Spin2_v45}
CON
  _clkfreq = 200_000_000
  QSIZE = 16
  QMASK = QSIZE - 1

  STRUCT job_t(LONG writerId, LONG payload)

VAR
  job_t jobs[QSIZE]                       ' shared hub queue of records
  LONG jhead, jtail
  LONG qlock  ' a hardware lock guards the multi-writer head advance

DAT
  writerStackA LONG 0[64]
  writerStackB LONG 0[64]

PUB main() | job_t j
  jhead := 0
```

continues on next page →

↪ continued from previous page

```
    jtail := 0
    qlock := LOCKNEW()  ' allocate one of the 16 hardware locks
    if qlock < 0
      DEBUG("no lock available")
      return

    ' TWO writers contend for the head -> lock needed
    cogspin(NEWCOG, writer(1), @writerStackA)
    cogspin(NEWCOG, writer(2), @writerStackB)

    ' Single consumer (this cog) drains; owns jtail, no lock to read.
    repeat
      if jtail <> jhead
        j := jobs[jtail]
        jtail := (jtail + 1) & QMASK
        DEBUG("job from writer ", udec(j.writerId), ...
              " payload ", udec(j.payload))

  PRI writer(id) | n
    n := 0
    repeat
      ' Enqueue is a read-modify-write on jhead shared by both
      ' writers -> bracket it with the lock.
      repeat until LOCKTRY(qlock)          ' spin-acquire
      if ((jhead + 1) & QMASK) <> jtail     ' not full
        jobs[jhead].writerId := id
        jobs[jhead].payload  := n
        jhead := (jhead + 1) & QMASK
      LOCKREL(qlock)  ' release on the single path out of the section
      n++
      waitms(5)
```

How this works. `LOCKNEW` allocates one of the P2's 16 hardware locks (returning `-1` if none are free — check it). Each writer, before touching `jhead`, spins on `REPEAT UNTIL LOCKTRY(qlock)` to capture the lock, does the whole enqueue — check-full, write the record, advance `jhead` — then `LOCKREL`s. Because only one writer holds the lock at a time, two writers can't both grab the same slot or both advance the head. The single consumer owns `jtail` and needs no lock to read. The lock protects exactly the shared read-modify-write, and nothing more.

▲ **Pitfall — release on every path out of the critical section.** Here there's one exit, so one `LOCKREL`. The moment you add an early `RETURN` or `ABORT` inside the locked section, you must release on *that* path too, or the lock is never freed and every other writer spins forever.

▲ **Pitfall — if you take two locks, always take them in the same order.** This recipe uses one lock. If a design ever needs two, every cog must acquire them in the same order (e.g. ascending by lock id); mismatched order is the classic deadlock. (The knowledge base lock model spells this out.)

♦ **Verify:** DEBUG prints `job from writer 1 ...` and `job from writer 2 ...` lines interleaved, each payload counting up per writer, with no duplicated or skipped slots. If two jobs ever land in the same slot or the head appears to jump, the enqueue ran without the lock — confirm both writers `LOCKTRY` before touching `jhead` and `LOCKREL` after.

Newer STRUCT Facilities (Spin2 v53 / v54)

Everything above runs on **v45**, the version that introduced `STRUCT`. Two later additions extend what a record can do, and each raises the version floor **only for the file that uses it** — R1–R4 are unchanged and still compile on v45:

- **`OFFSETOF` (v53)** — the byte offset of a member within a record, computed by the compiler instead of counted by you.
- **Member bitfields (v54)** — named bit ranges *inside* a `BYTE`/`WORD`/`LONG` member, so an entire record of named fields can occupy a single long.

▲ **Pitfall — a clean compile does not prove your toolchain is new enough.** The v54 bitfield syntax parses unconditionally; there is no version table standing behind it, so an older-than-you-think compiler will not necessarily complain. `{Spin2_v54}` is a declaration of intent, not an enforced gate. State the version you mean anyway — the next person to read your file has no other way to know.

Recipe R5 — A Whole Record in One Long

Use it when the payload genuinely fits in 32 bits — a command with a small opcode and argument, a status word, a mode plus flags. Pack the named fields into one long and the record can be published with a single atomic store: no lock, no separate sequence counter, and the torn read stops being something you avoid and starts being something that cannot happen.

```
{Spin2_v54}
CON
  _clkfreq = 200_000_000

  ' The WHOLE command, packed into one long. The nameless form means the
  ' struct IS the backing long - no intermediate member name to write.
  STRUCT cmd_t(LONG.opcode[3..0].arg[19..4].seq[31..20])

VAR
  cmd_t cmd   ' the entire shared command block - exactly one long

DAT
  workerStack LONG 0[64]

PUB main() | i, cmd_t staged
  cmd := 0  ' whole-struct zero
  cogspin(NEWCOG, worker(), @workerStack)

  repeat i from 1 to 5
    staged.opcode := i                  ' build it PRIVATELY, field by field
    staged.arg    := i * 100
    staged.seq    := i                  ' the sequence rides INSIDE the long
    cmd := staged                       ' publish: ONE whole-struct store
    waitms(50)
```

continues on next page →

↪ continued from previous page

```
    DEBUG("all commands published")

  PRI worker() | cmd_t snap, lastSeq
    lastSeq := 0
    repeat
      snap := cmd                         ' snapshot: ONE whole-struct load
      if snap.seq <> lastSeq              ' new command? ask the SNAPSHOT
        lastSeq := snap.seq
        DEBUG("worker exec op=", udec(snap.opcode), ...
              " arg=", udec(snap.arg), " seq=", udec(snap.seq))
```

How this works. `STRUCT cmd_t(LONG.opcode[3..0].arg[19..4].seq[31..20])` declares one `LONG` carrying three named fields — a 4-bit opcode, a 16-bit argument, and a 12-bit sequence number, 32 bits exactly. This is the **nameless form**: the member has no name of its own, so the struct *is* the long and you reach the fields directly (`cmd.opcode`, not `cmd.member.opcode`). The sequence number rides *inside* the same long as the payload, which is the trick that removes R3's separate counter: publishing the command and signalling "a new one arrived" become the same single write.

But the atomicity does **not** come from fitting in a long. It comes from how you write it. `staged.opcode := i` is a read-modify-write of the whole backing long, so the writer builds the record in a **private local** and then publishes it with one whole-struct assignment, `cmd := staged`. The reader does the mirror image: `snap := cmd` takes a single-load snapshot, and every field it examines afterwards comes from that snapshot, never from the shared copy. One store out, one load in — nothing else can straddle an update.

▲ **Pitfall — one long is not automatically one store.** Writing `cmd.opcode`, then `cmd.arg`, then `cmd.seq` *directly into the shared record* is three separate read-modify-writes of that long, and a reader can catch it between any two of them — the exact torn read you packed the record to avoid. Fitting in one long is what makes an atomic publish *possible*; staging the record privately and storing it in **one** statement is what makes it *happen*. The read side has the same trap: snapshot once, then read fields from the snapshot.

▲ **Pitfall — the compiler checks the bit boundaries, not your intent.** A bit range that overruns its member (bit 8 of a `BYTE`, bit 32 of a `LONG`) is rejected. Fields that *overlap* each other are not — no disjointness check is performed, so two fields silently sharing bits will compile and quietly corrupt each other. Lay the bit budget out once and add it up.

★ **Tip — compare this with R3.** The latest-wins mailbox needed three longs of command, plus a sequence long, plus an ack long, plus the discipline of writing them in the right order. When the payload fits in 32 bits, R5 is the same idea made smaller and stricter: the payload and its sequence number are one atomic write, and the ordering rule has nowhere left to go wrong. When the payload *doesn't* FIT — a three-long sensor record — you are back to R2, R3, or R4, and the ordering discipline is back on you.

♦ **Verify:** DEBUG prints five `worker exec op=N arg=M seq=N` lines with N running 1..5 in order and `arg` equal to `op * 100`, then `all commands published`. Every line's `arg` must match its `op` — an `arg` that belongs to a *different* `op` means the record reached the shared long field-by-field instead of being staged and stored once.

Recipe R6 — Safe Raw Addressing with OFFSETOF

Use it when you have to leave dot-notation behind — a packed header arriving off a wire, a buffer handed to inline PASM, a region the streamer fills — but you still want the record's layout to be the compiler's job rather than yours.

```
{Spin2_v53}
CON
  _clkfreq = 200_000_000

  ' A packed 8-byte header, laid out exactly as it arrives on the wire.
  STRUCT hdr_t(LONG magic, WORD length, BYTE kind, BYTE flags)

VAR
  BYTE packet[64]   ' raw bytes, as if just received
  ^hdr_t ph   ' a typed view over those same bytes

PUB main() | payload
  [ph] := @packet   ' aim the view at the buffer - brackets write the POINTER

  ' Fill the header through RAW addressing - the way low-level code must -
  ' but with OFFSETOF supplying each offset instead of hand-counted numbers.
  LONG[@packet + OFFSETOF(hdr_t.magic)]  := $5032_5041
  WORD[@packet + OFFSETOF(hdr_t.length)] := 40
  BYTE[@packet + OFFSETOF(hdr_t.kind)]   := 7
  BYTE[@packet + OFFSETOF(hdr_t.flags)]  := %0000_0011

  ' Read the SAME bytes back by name, through the struct view.
  DEBUG("magic=", uhex_long(ph.magic), " length=", udec(ph.length), ...
        " kind=", udec(ph.kind), " flags=", uhex(ph.flags))

  ' The offsets the compiler computed - so you never hand-count them.
  DEBUG("offsets  magic=", udec(OFFSETOF(hdr_t.magic)), ...
        " length=", udec(OFFSETOF(hdr_t.length)), ...
        " kind=", udec(OFFSETOF(hdr_t.kind)), ...
        " flags=", udec(OFFSETOF(hdr_t.flags)), ...
        "  sizeof=", udec(SIZEOF(hdr_t)))

  ' The payload begins right after the header - SIZEOF says where.
  payload := @packet + SIZEOF(hdr_t)
  BYTE[payload] := $FF
  DEBUG("payload starts at +", udec(payload - @packet))
```

How this works. `OFFSETOF(hdr_t.length)` returns that member's byte offset within the record — `4` here, because `magic` is a `LONG` and records pack with no padding. So low-level code can address the buffer *by name*: `WORD[@packet + OFFSETOF(hdr_t.length)]` writes the length field without anyone typing a `4`. Aim a struct pointer at the same bytes and you can read them back by name as well, through `ph.length`. `SIZEOF(hdr_t)` answers the companion question — where the header ends and the payload begins.

The value here isn't brevity; `4` is shorter than `OFFSETOF(hdr_t.length)`. The value is what happens *later*. Widen `magic`, insert a field, reorder two members, and every hand-counted offset in your code becomes silently wrong — the writes still land, just in the wrong bytes. The `OFFSETOF` form simply keeps being right. It is what lets a record stay a record even where you have to touch it as raw memory.

★ **Tip — brackets write the pointer, the bare name writes the record.** `[ph] := @packet` aims the pointer at the buffer. Plain `ph :=` without the brackets means "assign to the record it points at," and the compiler will reject it — the same bracket rule as the child-object pitfall below.

▲ **Pitfall — `OFFSETOF` is not legal everywhere.** Like `SIZEOF`, it is restricted to `DAT`, `VAR`, `PUB`, and `PRI` blocks. Using it in a `CON` or `OBJ` block is a compile error.

◆ **Verify:** DEBUG prints `magic=50325041 length=40 kind=7 flags=03`, then `offsets magic=0 length =4 kind=6 flags=7 sizeof=8`, then `payload starts at +8`. Now do the real TEST: add a member to `hdr_t` ahead of `length` and re-run. The offsets and the payload start move on their own, the raw writes still land in the right bytes, and you changed nothing but the declaration.

Adapt It / Going Further

The recipes are the building blocks; a few moves generalize them:

- **A deque is a ring with both ends live.** R2's ring moves only forward (producer at head, consumer at tail). Let *both* ends PUSH and POP — advance or retreat head and tail — and you have a double-ended queue. It needs no new primitive, just the same index discipline applied at both ends (and a lock if more than one cog writes a given end).
- **Records cross object boundaries too.** A child object can export a STRUCT type (v49+); a parent imports it as `object.struct_t`. When a child *method* returns a record *pointer*, receive it with bracket notation — see the Pitfalls — or a large record trips a compile error.
- **The newer facilities compose with the older recipes.** Nothing about R5's packed record is confined to a mailbox: make R2's ring carry single-long elements and every slot becomes an atomic publish in its own right, with the index still ordering the stream. And `OFFSETOF` is what lets any of these records be handed to inline PASM without a magic number in sight.
- **Choosing the structure is the design step this note doesn't cover.** Latest-wins vs. queue vs. published telemetry, and copy vs. reference for one-to-many fan-out, are *contract decisions* — they shape how your cogs depend on each other, and some (like reference-counted fan-out) are hard to reverse later. That reasoning is the **P2 Architect's Guide**'s job, backed by the knowledge base's data-flow-contracts model. This note gives you the code for a choice; the Guide helps you make it.

Pitfalls & Notes

▲ **Pitfall — a multi-field record is not atomic; publish it with one long.** The reader can catch a record mid-update. Always write the fields first, then flip a single long (an index or a sequence counter) that makes them visible. That one long is the atomic hand-off.

▲ **Pitfall — publish the index/sequence LAST.** The correctness of every lock-free recipe here depends on the order: fields first, publish long last. Reverse it and readers see partial data.

▲ **Pitfall — one writer per index means no lock; several writers means a lock.** A single owner of an index long can advance it safely without a lock. Two cogs advancing the same index is a read-modify-write race — guard it (R4).

▲ **Pitfall — a record that fits in one long still needs a one-store publish.** Packing named bitfields into a single long (R5) makes an atomic hand-off *possible*, not automatic: each field write is a read-modify-write of that long, so three of them are three stores a reader can land between. Stage the record in a local, publish it with one whole-struct assignment, and snapshot it with one load on the way back in.

▲ **Pitfall — release the lock on every exit path.** An early `RETURN`/`ABORT` inside a locked section must release first, or the lock leaks and other cogs hang. And free the lock with `LOCKRET` when the program is done with it.

▲ **Pitfall — receiving a record pointer from a child object needs brackets.** When a method in a *child object* returns `^structName`, receive it as `[pRec] := child.method()`. Without the brackets, a record larger than 15 longs is treated as a full-struct return and fails to compile — even though only a pointer is passed. (Within one object file this isn't needed.)

■ **Hardware — the P2 has 16 locks.** `LOCKNEW` hands out one of 16 hardware locks and returns `-1` when they're exhausted; always check. Return locks with `LOCKRET` so they can be reused.

★ **Tip — keep the locked section tiny.** Hold a lock only for the shared read-modify-write, never across `DEBUG()` or `waitms`. The shorter the section, the less other cogs stall.

Conclusion

Spin2's `STRUCT` gives the P2 first-class records: named, typed, packed, copied and sized as a unit. Within a cog that's simply cleaner code. Across cogs, one fact organizes everything — a single long is atomic, a multi-field record is not — so every safe hand-off writes the record first and publishes it with one long last. A single owner of that long needs no lock (the ring buffer, the latest-wins mailbox); several writers of it need one hardware lock (the multi-writer queue). Those three shapes cover most cog-to-cog data flow. And when the payload is small enough, member bitfields let the record *become* that single long, so the publish and the payload are the same atomic write — provided you stage it privately and store it once, because a long that you fill a field at a time is still three stores. Which shape a given seam should use — and whether to copy or share — is a design decision the **P2 Architect's Guide** exists to help you make; this note gives you the worked code once you've made it.

Resources

- **Example library** — every recipe as a standalone, `pnut_ts`-compiled program, published as `P2AN007-src-<date>.zip` beside this note:
  - R1 — `in-cog-record.spin2`
  - R2 — `spsc-ring-buffer.spin2`
  - R3 — `latest-wins-mailbox.spin2`
  - R4 — `locked-multiwriter-queue.spin2`
  - R5 — `single-long-packed-record.spin2`
  - R6 — `packed-header-offsets.spin2`
- **P2 Architect's Guide** (a companion P2 Knowledge Base publication) — how to *choose* a data-flow contract (which structure, copy vs. reference, and why); this note implements those choices.
- **P2AN005 — Cooperative Multitasking with Spin2 TASK Methods** (a companion P2 Knowledge Base publication) — when the cogs sharing these structures are cooperative tasks in one cog.

References

1. *Spin2 Language Documentation*, v45 and later (Chip Gracey, Parallax Inc.) — the `STRUCT` facility (declaration, typed members, arrays, nesting, `SIZEOF`, pointers, and the v49 cross-object export/import), the hardware lock methods `LOCKNEW` / `LOCKTRY` / `LOCKREL` / `LOCKRET`, and the two later extensions used here: `OFFSETOF` (v53) and member bitfields, named and nameless forms (v54).
2. P2 Knowledge Base data-flow-contracts model — the named cog-to-cog contracts (latest-wins mailbox, ring buffer, published telemetry, fan-out) this note implements; the *choice* among them is an Architect's Guide topic.

Revision History

- **v1.0.0** (July 2026) — initial release for community review. Six runnable recipes, each `pnut_ts -d`-compiled: an in-cog record and array, a lock-free ring buffer, a latest-wins mailbox, a lock-guarded multi-writer queue, a whole record packed into one atomically-published long (`{Spin2_v54}` member bitfields), and computed member offsets for raw addressing (`{Spin2_v53} OFFSETOF`). Recipes R1–R4 need only `{Spin2_v45}`, so a compiler predating the newer facilities still builds them. Every cross-cog claim is **confirmed on real P2 silicon**, with two cogs actually contending: each discipline was measured against a deliberately-broken version of itself, and the broken version was required to fail before the result was accepted. Implementation-only by design; the contract-choice reasoning stays with the P2 Architect's Guide.

Copyright & License

Acknowledgments

Parallax Inc. and Chip Gracey — for the Propeller 2 silicon and the Spin2 language, including the STRUCT facility and the hardware lock model this note applies.